



Urban Heat & Health Regression in Singapore

Go Pipeline + AI Prompting Learning Journey

R^2

0.5978

RMSE

2.7997

MAE

2.5627



Objective: Estimate heat-related health risk across Singapore districts using temperature and demographic data. Outcome: a **moderate-fit baseline model** delivered through a reproducible Go data pipeline.

Why District-Level Heat Risk Estimation Matters

The Problem

Singapore's urban density amplifies heat island effects. Rising temperatures disproportionately harm elderly and lower-income communities, yet risk is typically modeled city-wide — masking local variation.

Project Goals

- Integrate air temperature, demographic, and health outcome data at district level
- Build an explainable baseline regression model in Go
- Demonstrate a reproducible, documented pipeline suitable for extension

- District-level granularity enables targeted public health interventions — identifying which neighborhoods face the highest combined temperature and vulnerability burden, rather than averaging risk across the city.



Data Sources & Coverage

Dataset	Rows	Format
Air Temperature	356	JSON → CSV
Demographics	15	CSV
Health Outcomes	15	CSV

Scope: Single city (Singapore), single year (2024). All 15 districts retained after cleaning — zero data loss across all three sources.

Key Variables

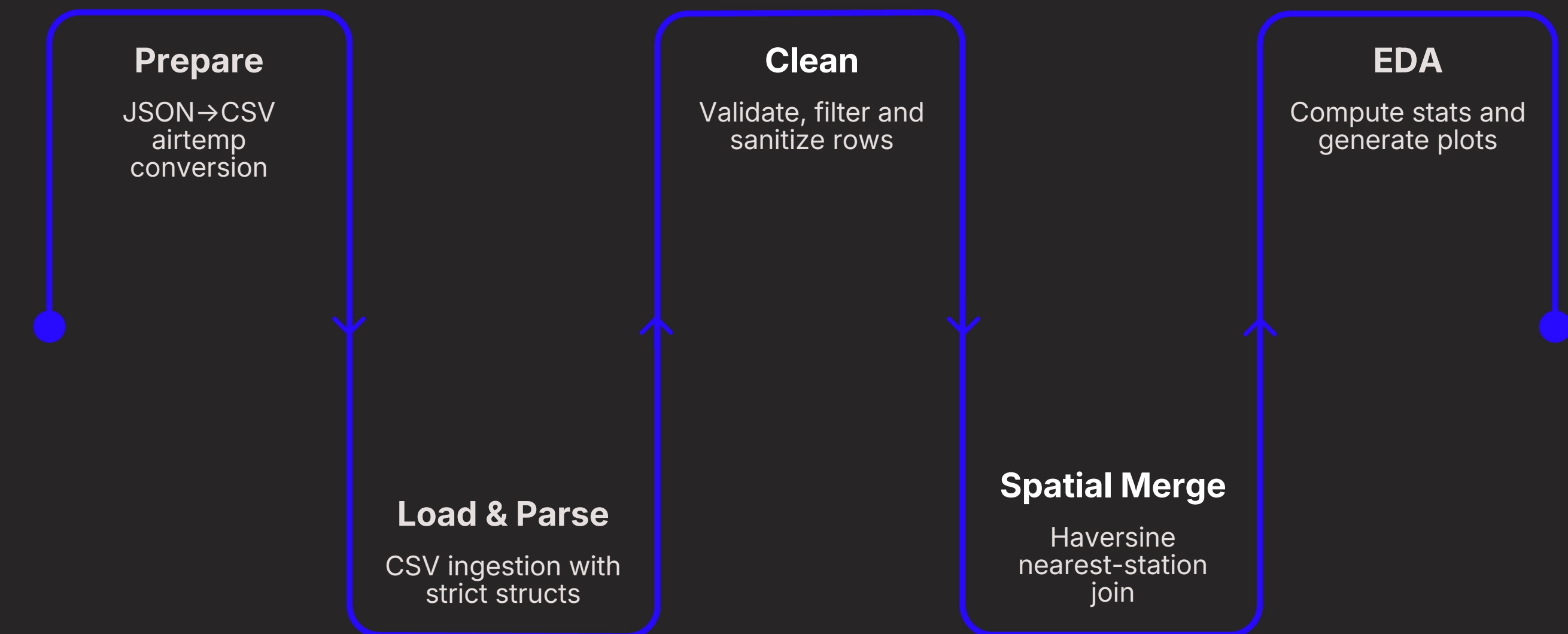
temperature_c
Mean 28.81°C

elderly_share
Mean 11.1%

heat_cases / 100k
Mean 37.65

📄 The temperature dataset is larger (356 station-level rows) and is aggregated spatially to match the 15 district-level demographic and health records. Perfect data retention across all sources is a notable pipeline quality result.

Go Pipeline Architecture



Strict Typed Structs

All data modeled with explicit Go structs for compile-time safety



Explicit Error Handling

Every I/O and parse call returns checked errors — no silent failures



Reproducible CLI Workflow

Seeded train/test split; deterministic outputs from single command



Choosing Go over Python forced explicit type discipline and structured error paths — making the pipeline more verbose but far more robust. This was an intentional pedagogical choice to deepen Go fluency alongside data engineering practice.

Data Preparation & Feature Engineering

Cleaning Rules

- **Required fields** (station ID, coordinates, timestamp, value) trigger row skip if missing
- **Optional fields** tolerate nulls; defaults applied where appropriate
- Result: **zero dropped rows** across all three datasets

Spatial Merge Strategy

- Nearest temperature station assigned to each district centroid via **Haversine distance**
- Maximum distance threshold enforced to prevent spurious matches

Output Schema & Reproducibility

Merged CSV

One row per district with temperature, demographics, and health outcome joined

Seeded Split

Fixed random seed ensures identical train/test partitions on every run

Documented Schema

Column names and units recorded in pipeline header comments



The Haversine-based spatial merge is the most technically demanding step — matching 356 station observations to 15 district centroids. The distance threshold prevents distant stations from being incorrectly assigned, protecting model validity.

EDA Highlights

Descriptive Statistics

Variable	Min	Max
temperature_c	28.2°C	29.6°C
elderly_share	—	—
heat_cases/100k	—	—
population (mean)	108,667	

Key insight: Temperature range is narrow (1.4°C spread) yet heat case rates vary substantially across districts — suggesting demographics drive outcome variance more than temperature alone.

Temperature Distribution

Near-normal histogram centered at 28.81°C; low variance confirms urban heat island homogeneity

District Heat Cases

Bar chart of mean heat_cases/100k by district reveals pronounced inter-district inequality

Rows by Year

All 356 temperature records fall within 2024 — confirming single-year temporal scope

❏ The narrow temperature range (28.2–29.6°C) is analytically important: it limits temperature's explanatory power in regression. This makes the demographic covariates — especially elderly_share — relatively more influential, which the model coefficients confirm.

Regression Model & Results

Model Formula

$$\text{heat_cases} = \beta_0 + \beta_1 \cdot \text{temperature_c} + \beta_2 \cdot \text{population} + \beta_3 \cdot \text{elderly_share} + \beta_4 \cdot \text{income_index}$$

OLS implemented from scratch in Go (no external ML library).

Performance Metrics

Coefficient Table

Feature	Coefficient
elderly_share	287.8685 ★
temperature_c	6.0491
income_index	—
population	—
intercept	included

Test set N=3 — interpret metrics cautiously. Largest residual: district S115 at **−4.1446**.

R² = 0.5978

~60% variance explained

RMSE = 2.7997

Avg. prediction error

MAE = 2.5627

Median absolute error

📌 The elderly_share coefficient (287.87) is by far the dominant predictor, consistent with known epidemiology — older populations face greater heat vulnerability. Temperature's positive contribution (6.05) is theoretically expected but limited by the narrow observed range.

Limitations, Risks & Next Steps

Current Limitations

Small Sample

Only 15 district records after merge; test set N=3 makes metrics volatile

Single City / Year

2024 Singapore only — no temporal or cross-city generalization

Coefficient Instability

OLS on small N risks large standard errors and sensitivity to outliers

Recommended Next Steps



Enrich Features

Add humidity, rainfall, green cover (NDVI), and building density



Expand Scope

Multi-year data and additional cities to improve generalization



Compare Models

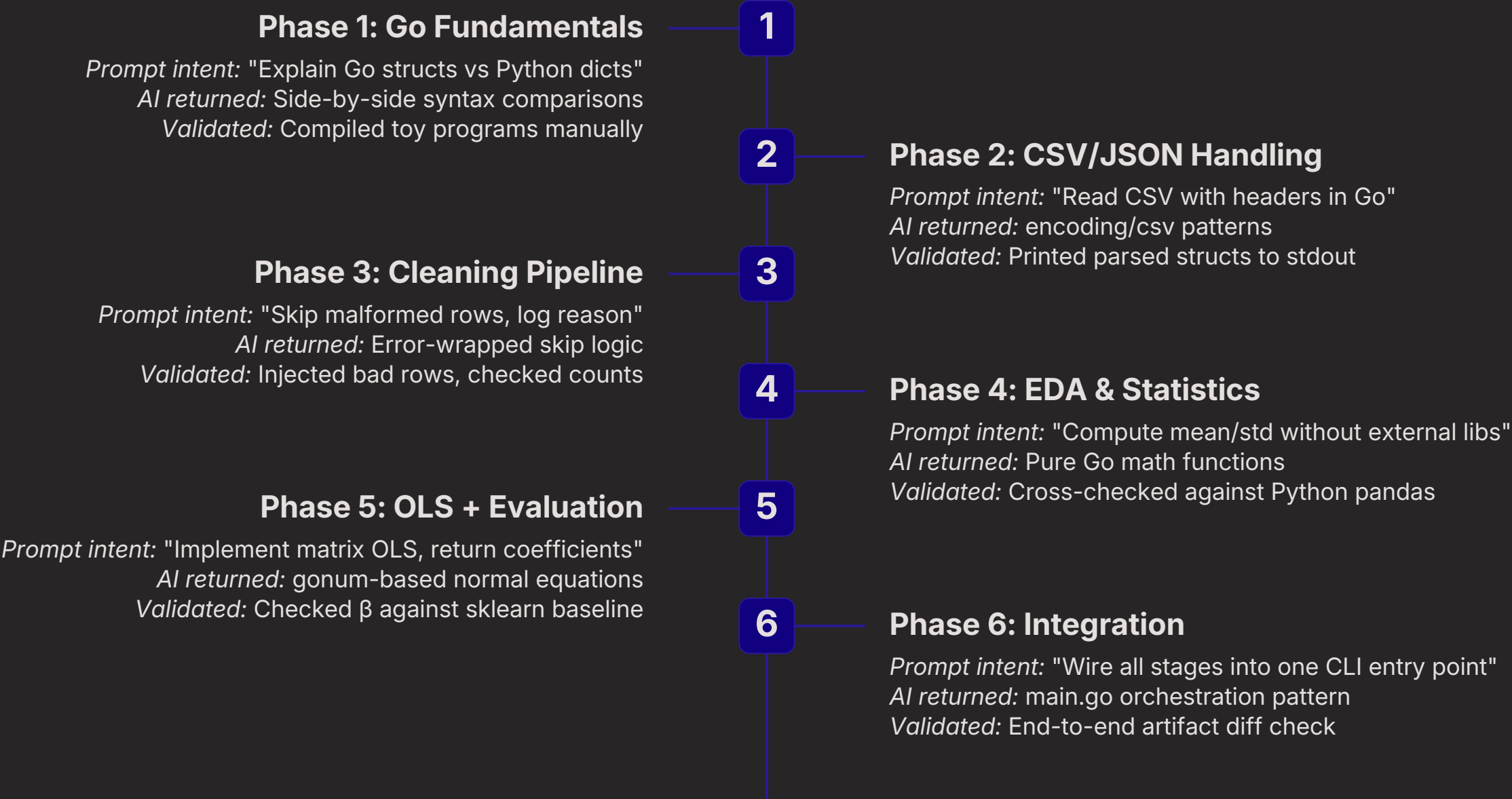
Benchmark OLS against Ridge, Lasso, and tree-based regressors



The N=3 test set is the project's most critical limitation — it means reported RMSE and R^2 should be treated as indicative, not definitive. Future iterations must prioritize data breadth before model complexity. District S115's residual of -4.15 warrants manual investigation.

AI Prompt Engineering Journey

Prompting evolved systematically from broad conceptual learning to precise, task-specific implementation across six phases.



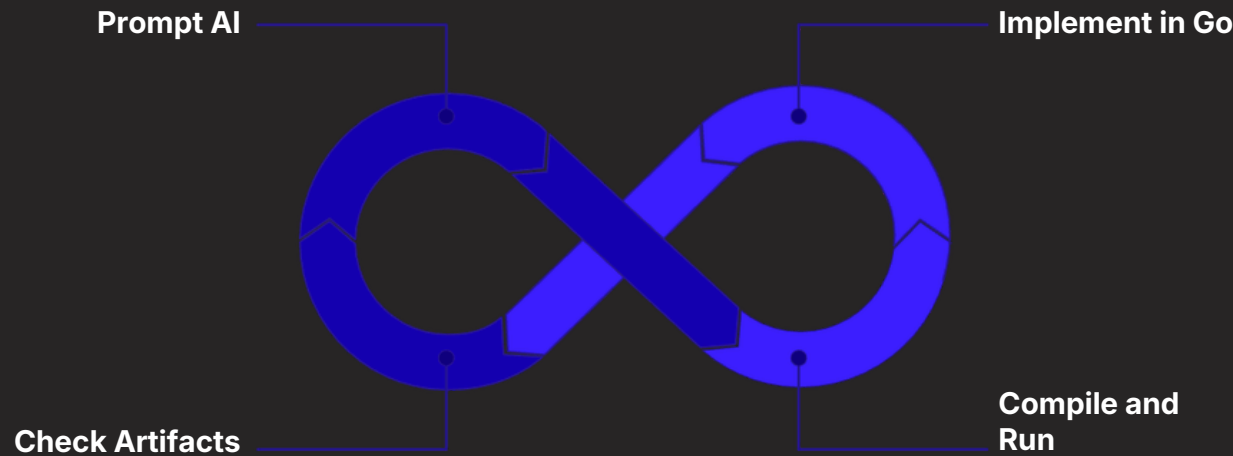
The key insight from this journey: early prompts were exploratory ("explain the concept"), while later prompts became highly constrained ("implement X with these types, returning these errors, compatible with this struct schema"). Prompt specificity correlates directly with output usability.

Responsible AI Use, Best Practices & Takeaways

AI as Tutor, Not Author

Every AI-generated code block was read, understood, modified, and manually validated before integration. AI accelerated learning — it did not replace engineering judgment.

Validation Loop



Prompt Best Practices Learned

01

Provide Context & Constraints

Specify language, library restrictions, and existing struct schemas upfront

0

2

Ask for Comparisons

Requesting Python vs Go side-by-side accelerated conceptual transfer

0

3

Request Step-by-Step Outputs

Decomposed prompts yielded more testable, auditable code segments

0

4

Require Validation Criteria

Asking "how would I verify this is correct?" surfaced edge cases proactively



Final Takeaway: Disciplined prompting with manual verification is a force multiplier — it compressed weeks of Go learning into a focused project cycle while maintaining full intellectual ownership of every design and implementation decision.